VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

MACHINE LEARNING - SEMESTER 242

ASSIGNMENT 2

Lecturer: NGUYỄN AN KHƯƠNG

Class: CC01

Students: Đặng Ngọc Phú - 2252617
Phùng Gia Minh Khôi - 2252381
Nguyễn Lê Khải Trọng - 2252850

HO CHI MINH CITY, MAR 2025

Contents

Team Members and Task Distribution

Team Member	Student's ID	Contributions
Đặng Ngọc Phú	2252617	Do MLP
Phùng Gia Minh Khôi	2252381	Do HMM
Nguyễn Lê Khải Trọng	2252850	Do reduced dimension

1 Github link

https://github.com/Chilly-On/ML_Researchers

2 Introduction

TED Talks have revolutionized the way we exchange ideas, offering a platform where innovators, educators, and thought leaders share transformative insights on topics ranging from science and technology to art and social issues. Much like the automotive industry, which is populated by globally recognized brands each distinguished by unique models, manufacturing years, pricing, performance metrics, and design philosophies, TED Talks are characterized by their diverse themes, speaker backgrounds, presentation styles, and cultural contexts.

This project harnesses Natural Language Processing (NLP) techniques to delve into the vast repository of TED Talks transcripts, aiming to extract meaningful patterns and trends that underpin these influential speeches. By examining various attributes—such as the speaker's professional background, the publication year of the talk, sentiment, key topics discussed, and audience engagement metrics—we seek to draw parallels between the evolution of public discourse and the dynamic nature of modern communication.

The study not only explores the linguistic nuances and stylistic elements that make each talk unique but also investigates how these factors collectively contribute to the overall impact of the presentation. In doing so, it provides a framework for understanding how ideas are communicated and received across different segments of society. The insights derived from this analysis are expected to shed light on the broader cultural and intellectual trends that influence our contemporary world, ultimately enhancing our comprehension of persuasive communication and its role in shaping public opinion.

3 Flowchart for training model

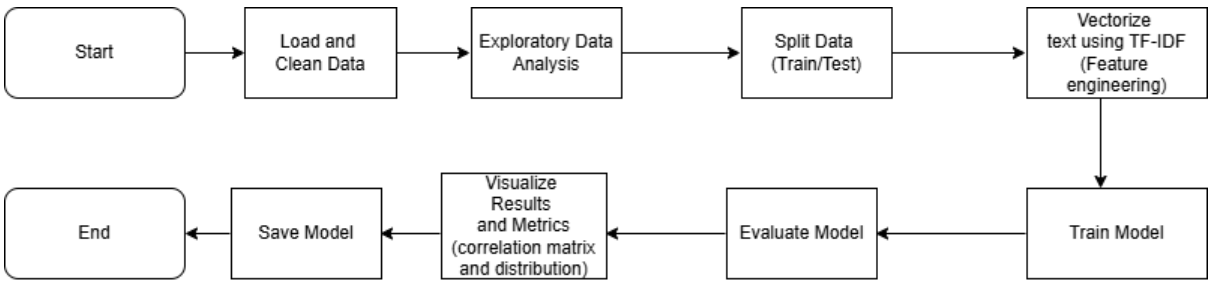

Figure 1: General flowchart for training model

Data Preparation:
+ Collect raw data from TED Talks Transcripts for NLP.
+ Extract the transcription column and remove any missing values from the dataset.
+ Transform the text to lowercase and eliminate any numerical digits and punctuation marks.
Data Preprocessing:
+ Feature Extraction: Use the TfidfVectorizer to turn cleaned text into numbers by calculating TF-IDF scores. It considers up to 5000 features, including both single words (unigrams) and pairs of words (bigrams).
+ Transformation: The vectorizer creates a sparse matrix, which we then convert into a dense array for easier handling.
+ DataFrame Creation:
*Build a new DataFrame where each column represents a word or n-gram feature.
*Add a target column (currently set to None) to set the stage for later modeling steps.
Model Evaluation:
It helps in comparing models and fine-tuning hyperparameters during the development phase by visualizing correlation matrix and distribution.
Model Selection To Training :
+ The training involves tuning model parameters through optimization techniques to minimize error or maximize performance.
Final Evaluation:
+ Evaluate the final result after the training to conduct a final evaluation on a hold-out test set that was not used during training or validation. This provides an unbiased estimate of the model's real-world performance before deployment.

4 Code for training model

Clean Data Code & Vectorize text using TF-IDF

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd
import re

# Select the transcription column and clean the null in the data
train_data = pd.read_csv(f"C:/Users/DELL/Desktop/Machine learning/ML_Researchers-data/
    data/raw/ted_talks_en.csv")
df = pd.DataFrame(train_data)
print("\nData converted to DataFrame:")
df = df[['transcript']]
df = df.dropna(subset=['transcript'])
print(df.head())

def clean_text(text):
    text = text.lower()  # Convert to lowercase
    text = re.sub(r'\d+', '', text)  # Remove numbers
    text = re.sub(r'[^\w\s]', '', text)  # Remove punctuation
    text = text.strip()
    return text

df['cleaned_transcript'] = df['transcript'].apply(clean_text)
```

```

df = df[['cleaned_transcript']]        # Only keep cleaned transcript
print(f"Language en:")
print(df.head())

# Split data into features and target variable
X = df['cleaned_transcript']

# Vectorizing text using TF-IDF
vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1, 2))  # Consider unigrams and bigrams
X_vectorized = vectorizer.fit_transform(X)

# Convert to dense array
X_dense = X_vectorized.toarray()

# Create DataFrame with words as columns
df_tfidf = pd.DataFrame(X_dense, columns=vectorizer.get_feature_names_out())
print(X_vectorized.shape)
print(df_tfidf.shape)
df_tfidf['target'] = None

# Display a sample
print(df_tfidf.head())
```

Model Code

```
# Import libraries
# Import libraries
import sys
import os
os.chdir(r"C:\Users\Admin\Downloads\ML_Researchers")
import torch

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
import numpy as np
from sklearn.model_selection import train_test_split

from src.data import load_raw_data, preprocess_data, save_processed_data
from src.features import create_features
from src.models import train_models, evaluate_models, tune_hyperparameters, save_models, train_MLP, MLP
from src.visualization import classDistVisualize

# Step 1: Load raw data
print("Loading raw data...")
raw_data = load_raw_data("data/raw/ted_talks_en.csv")
print("Raw Data Sample:")
print(raw_data.head())

# Step 2: Define target based on views
print("\nDefining target variable...")
median_views = raw_data['views'].median()
raw_data['target'] = raw_data['views'].apply(lambda x: 1 if x > median_views else 0)
```

```
print("Target Distribution:")
print(raw_data['target'].value_counts())

# Step 3: Preprocess the data
print("\nPreprocessing data...")
processed_data = preprocess_data(raw_data, text_column='transcript')
print("Processed Data Sample:")
print(processed_data.head())

# Save processed data
save_processed_data(processed_data, "data/processed/processed_ted_talks_en.csv")
print("\nProcessed data saved to 'data/processed/processed_ted_talks_en.csv'.")

# Step 4: Feature engineering (TF-IDF vectorization)
print("\nPerforming feature engineering...")
df_tfidf = create_features(processed_data, text_column='transcript', max_features=5000,
    ngram_range=(1, 2))
print("TF-IDF Data Sample:")
print(df_tfidf.head())

# Save vectorized data
df_tfidf.to_csv("data/processed/vectorized_data.csv", index=False)
print("\nVectorized data saved to 'data/processed/vectorized_data.csv'.")

# Step 5: Split data into train, validation, and test sets
print("\nSplitting data into train, validation, and test sets...")
X = df_tfidf.drop(columns=['target'])  # Features
y = df_tfidf['target']  # Target variable

# Split into train (80%) and test (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state
    =42)

# Split train into train (80%) and validation (20%)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.2,
    random_state=42)

print(f"Train set shape: {X_train.shape}")
print(f"Validation set shape: {X_val.shape}")
print(f"Test set shape: {X_test.shape}")

# visualize the distribution of the variable on pca
train_data = [np.array(X_train), np.array(y_train)]
eval_data = [np.array(X_val), np.array(y_val)]
classDistVisualize('pca', train_data, eval_data)

# visualize the distribution of the variable on tsne
classDistVisualize('tsne', train_data, eval_data)

# Step 6: Save test data for submission.ipynb
print("\nSaving test data for submission.ipynb...")
test_data = pd.concat([X_test, y_test], axis=1)
test_data.to_csv("data/processed/test_data.csv", index=False)
print("Test data saved to 'data/processed/test_data.csv'.")

from src.models.train_model import tune_reduced_dimension
from sklearn.ensemble import RandomForestClassifier
import pandas as pd

```

```python
# Step 7: Tune with reduced dimension
param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [None, 10]
}

print("\nTuning hyperparameters (with dimensionality reduction)...")
best_model, best_params = tune_reduced_dimension(
    X_train, y_train, X_val, y_val,
    model_class=RandomForestClassifier,
    param_grid=param_grid,
    pca_components=2
)

# ---  Retrain full data train + val ---
X_full = pd.concat([X_train, X_val], axis=0)
y_full = pd.concat([y_train, y_val], axis=0)

print("\nRetraining best model on full train+val set...")
best_model.fit(X_full, y_full)

# Save reduced dimension model:
import joblib
joblib.dump(best_model, "models/trained/rf_pca_tuned.pkl")
print("Saved final tuned model to models/trained/rf_pca_tuned.pkl")

# Step 8: Hyperparameter tuning using GridSearchCV
print("\nPerforming hyperparameter tuning...")
best_models = tune_hyperparameters(X_train, y_train, X_val, y_val)

# Step 9: Evaluate the best models on the validation set
print("\nEvaluating best models on the validation set...")
val_results = evaluate_models(best_models, X_val, y_val)
print("Validation Set Evaluation Results:")
for model, metrics in val_results.items():
    print(f"{model}: {metrics}")

# Step 10: Train the best model on the entire train set (train + validation)
print("\nTraining the best model on the entire train set...")
best_model_name = max(val_results, key=lambda k: val_results[k]['accuracy'])
best_model = best_models[best_model_name]

# Combine train and validation sets
X_train_full = pd.concat([X_train, X_val])
y_train_full = pd.concat([y_train, y_val])

# Train the best model
best_model.fit(X_train_full, y_train_full)

# Step 11: Train and evaluate on MLP model
model = train_MLP(X_train, y_train, X_val, y_val, X_test, y_test, input_size=4999,
    epochs=300, batch_size=32, lr=0.00001)

# Step 12: Evaluate the best model on the test set
print("\nEvaluating the best model on the test set...")
test_results = evaluate_models({best_model_name: best_model}, X_test, y_test)
print(f"\n{best_model_name} Test Set Evaluation Results:")
for model, metrics in test_results.items():
    print(f"{model}: {metrics}")
```

```
144
145 # save the model mlp to trained folder
146 print("\nSaving MLP model...")
147 torch.save(model, "models/trained/mlp.pt")
```